

Contents

1	Architecture Model Standard v2.0	2
1.1	Table of Contents	2
1.2	1. Introduction	2
1.2.1	1.1 Purpose	2
1.2.2	1.2 Scope	3
1.2.3	1.3 Design Principles	3
1.2.4	1.4 Terminology	3
1.3	2. Conformance	4
1.4	3. Concepts	4
1.4.1	3.1 Architecture as Code	4
1.4.2	3.2 The Model Pipeline	4
1.4.3	3.3 Entity-Relationship Model	5
1.4.4	3.4 Functional Blocks (F-blocks)	5
1.4.5	3.5 Layers	5
1.4.6	3.6 Status Lifecycle	5
1.5	4. Schema	5
1.5.1	4.1 File Format	5
1.5.2	4.2 Meta	6
1.5.3	4.3 Entities	7
1.5.4	4.4 Relationships	22
1.5.5	4.5 Base Entity Properties	24
1.6	5. Recursive Decomposition	25
1.6.1	5.1 Overview	25
1.6.2	5.2 Mechanism	25
1.6.3	5.3 Rules	25
1.6.4	5.4 Example	25
1.7	6. Reality Manifest	26
1.7.1	6.1 Overview	26
1.7.2	6.2 Purpose	26
1.7.3	6.3 Manifest Structure	26
1.7.4	6.4 Per-File Detail	27
1.7.5	6.5 Per-F-block Detail	27
1.7.6	6.6 Pipeline	27
1.7.7	6.7 Manifest Hash	28
1.8	7. Domain Profiles	28
1.8.1	7.1 Overview	28
1.8.2	7.2 Activation	28
1.8.3	7.3 Available Profiles	28
1.8.4	7.4 Profile Composition	30
1.8.5	7.5 Extensions for Custom Profiles	30
1.9	8. LLM Integration Protocol	30
1.9.1	8.1 Overview	30
1.9.2	8.2 Verbs	30
1.9.3	8.3 Context Compression	32
1.10	9. Validation Rules	32
1.10.1	9.1 Overview	32
1.10.2	9.2 Severity Levels	32
1.10.3	9.3 Rules	33
1.11	10. Examples	34
1.11.1	10.1 Software System	34
1.11.2	10.2 Mechanical Gearbox	37
1.11.3	10.3 Controls System (SCADA)	40

1.11.4 10.4 Electrical PCB Assembly	42
1.12 Appendix A: JSON Schema Reference	45
1.12.1 A.1 Top-Level Schema	45
1.12.2 A.2 Meta Schema	45
1.12.3 A.3 Base Entity Schema	46
1.12.4 A.4 Relationship Schema	46
1.12.5 A.5 Entity Collection Keys	47
1.13 Appendix B: Relationship Type Matrix	48
1.14 Appendix C: Changelog	48
1.14.1 v2.0 (2026-07-18)	48
1.14.2 v1.4 (2026-06-15)	49
1.14.3 v1.3 (2026-05-01)	49
1.14.4 v1.0 (2026-01-01)	49

1 Architecture Model Standard v2.0

Status: Draft Specification **Date:** 2026-07-18 **Authors:** Architecture Model Standard Contributors **License:** Apache 2.0

1.1 Table of Contents

1. Introduction
 2. Conformance
 3. Concepts
 4. Schema
 - 4.1 File Format
 - 4.2 Meta
 - 4.3 Entities
 - 4.4 Relationships
 - 4.5 Base Entity Properties
 5. Recursive Decomposition
 6. Reality Manifest
 7. Domain Profiles
 8. LLM Integration Protocol
 9. Validation Rules
 10. Examples
 - 10.1 Software System
 - 10.2 Mechanical Gearbox
 - 10.3 Controls System (SCADA)
 - 10.4 Electrical PCB Assembly
 - Appendix A: JSON Schema Reference
 - Appendix B: Relationship Type Matrix
 - Appendix C: Changelog
-

1.2 1. Introduction

1.2.1 1.1 Purpose

The Architecture Model Standard defines a universal, machine-readable format for describing the architecture of any engineered system. It provides a YAML-based schema that captures entities, relationships,

constraints, and behavioral specifications in a form suitable for both human review and automated processing.

The standard is designed to serve as the architectural spine for LLM-driven system engineering. By encoding architecture in a structured, validated format, it enables AI agents to load, query, reason about, and update architectural models with full traceability to source code and design artifacts.

1.2.2 1.2 Scope

This specification covers:

- The YAML schema for architecture model documents (Section 4).
- The 15 entity types and their field definitions (Section 4.3).
- The 16 relationship types and their semantics (Section 4.4).
- Recursive decomposition for systems-of-systems (Section 5).
- The Reality Manifest format for AST-derived ground truth (Section 6).
- Domain profiles for cross-domain modeling (Section 7).
- The LLM Integration Protocol for AI agent interaction (Section 8).
- Validation rules for model correctness (Section 9).

This specification does NOT cover:

- Implementation details of parsers, validators, or CLI tools.
- Specific LLM prompting strategies or agent architectures.
- Deployment or CI/CD integration patterns.

1.2.3 1.3 Design Principles

1. **Code-anchored.** Architecture models **MUST** be traceable to source code reality via AST-derived manifests. Models that drift from code are invalid.
2. **Domain-neutral.** The base schema applies to software, controls, mechanical, electrical, and hybrid systems. Domain profiles extend the base without modifying it.
3. **Machine-first, human-readable.** The format is optimized for automated parsing and LLM consumption while remaining accessible to human reviewers.
4. **Recursively decomposable.** Any component **MAY** be expanded into a full sub-model, enabling systems-of-systems at arbitrary depth.
5. **Validation-driven.** Every model **MUST** pass structural validation before it is considered conformant. Validation is deterministic and requires no LLM involvement.

1.2.4 1.4 Terminology

Term	Definition
Model	A single <code>.architecture-model.yaml</code> file conforming to this standard
Entity	A named, typed element within the model (component, interface, etc.)
Relationship	A typed, directed link between two entities
F-block	A functional block — a grouping of components realizing a capability
Reality Manifest	An AST-derived inventory of a codebase's actual structure
Domain Profile	A schema extension adding domain-specific types and properties
Sub-model	A model that decomposes a parent model's component

1.3 2. Conformance

The key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in RFC 2119.

A conformant implementation:

1. MUST parse any valid model document without error.
2. MUST reject model documents that violate the validation rules in Section 9.
3. MUST preserve all fields during round-trip serialization (parse → serialize → parse yields identical model).
4. MUST support all 15 entity types defined in Section 4.3.
5. MUST support all 16 relationship types defined in Section 4.4.
6. SHOULD support domain profiles as defined in Section 7.
7. MAY support the LLM Integration Protocol defined in Section 8.
8. MAY support recursive decomposition as defined in Section 5.

A conformant model document:

1. MUST be valid YAML 1.2.
 2. MUST contain a meta section with at least project and schema_version.
 3. MUST nest all entities under the entities key.
 4. MUST assign a unique id to every entity.
 5. MUST reference only existing entity IDs in relationships.
 6. MUST pass all validation rules defined in Section 9.
-

1.4 3. Concepts

1.4.1 3.1 Architecture as Code

An architecture model is a machine-readable description of a system’s structure, behavior, and constraints. Unlike diagrams or prose documents, an Architecture-as-Code model is:

- **Versioned** — stored in source control alongside the code it describes.
- **Validated** — checked for structural correctness by deterministic rules.
- **Anchored** — linked to code reality via file paths, line numbers, and AST-derived manifests.
- **Compressible** — sliceable into focused subsets for token-efficient LLM consumption.

1.4.2 3.2 The Model Pipeline

The standard prescribes a pipeline from source code to validated model:

```
Code --> [AST Scan] --> Reality Manifest --> [LLM Enrichment] --> Model --> [Validator] --> .architecture-model.yaml
```

1. **AST Scan.** A deterministic scanner analyzes source files to produce a Reality Manifest (Section 6). This step involves no LLM and produces ground truth.
2. **LLM Enrichment.** An LLM agent receives the manifest and produces a structured architecture model, adding semantic groupings, capability descriptions, and relationship typing that AST analysis alone cannot determine.
3. **Validation.** The model is validated against the schema and the manifest. Validation is deterministic and checks structural integrity, referential consistency, and manifest alignment (Section 9).

1.4.3 3.3 Entity-Relationship Model

The standard uses a typed entity-relationship model. Entities represent architectural elements (components, interfaces, constraints, etc.). Relationships represent typed, directed links between entities.

Every entity has a unique identifier, a human-readable name, a status, and an optional description. Entity types define additional fields specific to their domain (see Section 4.3).

Relationships are first-class elements with their own properties, including type, strength, and optional description. They connect entities via `from` and `to` references using entity IDs.

1.4.4 3.4 Functional Blocks (F-blocks)

An F-block is a logical grouping of components that together realize a capability. F-blocks provide a higher-level view of the system than individual components. Each capability entity declares an `f_block` identifier, and components reference this identifier to indicate which capability they contribute to.

F-blocks are the primary unit of slicing for LLM context compression. When an agent needs to reason about a specific capability, the model can be sliced to include only the F-block's components, their interfaces, and their transitive dependencies.

1.4.5 3.5 Layers

Layers represent architectural tiers — horizontal slices of the system organized by abstraction level or deployment concern. Components are assigned to layers, and layers define ordering to express dependency direction.

1.4.6 3.6 Status Lifecycle

Every entity carries a status field with one of four values:

- **ACTIVE** — The entity is currently part of the system.
- **PLANNED** — The entity is designed but not yet implemented.
- **DORMANT** — The entity exists but is not currently active.
- **DEPRECATED** — The entity is marked for removal.

Status transitions are not enforced by the schema but SHOULD follow the progression: **PLANNED** → **ACTIVE** → **DORMANT** or **DEPRECATED**.

1.5 4. Schema

1.5.1 4.1 File Format

A conformant model document:

1. MUST be a single YAML 1.2 file.
2. MUST use the filename `.architecture-model.yaml` unless explicitly configured otherwise.
3. MUST be encoded in UTF-8.
4. MUST contain exactly three top-level keys: `meta`, `entities`, and `relationships`.

Top-level structure:

```
meta:
  # Model metadata (Section 4.2)
entities:
  # Entity collections by type (Section 4.3)
```

`relationships:`

List of typed relationships (Section 4.4)

The `entities` key MUST be a mapping where each key is a plural entity type name (e.g., components, interfaces) and each value is a list of entity objects. Entity type keys that are absent or empty are permitted — they indicate no entities of that type exist in the model.

The `relationships` key MUST be a list of relationship objects.

1.5.2 4.2 Meta

The meta section provides model-level metadata.

Field	Type	Required?	Description
<code>schema_version</code>	string	Yes	Version of this standard the model conforms to. MUST be "2.0" for models conforming to this specification.
<code>project</code>	string	Yes	Human-readable project name.
<code>system</code>	string	No	System or subsystem name, if different from project.
<code>generated_at</code>	string (ISO 8601)	No	Timestamp of model generation.
<code>source_artifacts</code>	string[]	No	List of source file paths or glob patterns used to generate this model.
<code>manifest_hash</code>	string	No	SHA-256 hash of the Reality Manifest used during generation. Links the model to a specific code state.
<code>source_language</code>	string	No	Primary programming language of the source (e.g., python, typescript, c++).
<code>domain_profile</code>	string	No	Domain profile to apply. One of: software, controls, mechanical, electrical. Default: software. See Section 7.
<code>parent_model</code>	string	No	Relative path to the parent model file. Used for recursive decomposition (Section 5).
<code>refines_component</code>	string	No	Entity ID of the parent model's component that this sub-model decomposes. Used with <code>parent_model</code> .

Example:

`meta:`

```
schema_version: "2.0"
project: acme-web-platform
system: order-processing
generated_at: "2026-07-18T14:30:00Z"
source_artifacts:
```

```

- "src/**/*.py"
manifest_hash: "sha256:a1b2c3d4e5f6..."
source_language: python
domain_profile: software

```

1.5.3 4.3 Entities

This section defines all 15 entity types. Every entity type inherits the base properties defined in Section 4.5. The field tables below list only the type-specific fields; base fields are always available.

Entity collections are keyed by the plural form of the type name under the entities top-level key:

```

entities:
  actors: [...]
  capabilities: [...]
  behaviors: [...]
  interfaces: [...]
  constraints: [...]
  layers: [...]
  components: [...]
  systems: [...]
  data: [...]
  events: [...]
  resources: [...]
  environments: [...]
  quality_attributes: [...]
  decisions: [...]
  lifecycles: [...]

```

1.5.3.1 4.3.1 Actors An Actor is an external agent that interacts with the system. Actors initiate behaviors, consume interfaces, and represent the boundary between the system and its environment. Actors are not part of the system itself.

Type-specific fields:

Field	Type	Required?	Description
type	string	Yes	The kind of actor. One of: human, system, external-service.
goals	string[]	No	High-level goals this actor pursues through the system.

YAML example:

```

entities:
  actors:
    - id: ACT-1
      name: Site Administrator
      status: ACTIVE
      description: Manages platform configuration and user accounts.
      type: human
      goals:
        - "Configure system parameters"

```

- "Manage user roles and permissions"
 - id: ACT-2
 - name: Payment Gateway
 - status: ACTIVE
 - type: external-service
 - goals:
 - "Process payment transactions"
-

1.5.3.2 4.3.2 Capabilities A Capability is a functional block (F-block) that the system provides. Capabilities represent what the system does at a business or mission level, independent of implementation. Components realize capabilities.

Type-specific fields:

Field	Type	Required?	Description
f_block	string	No	F-block identifier for grouping. Components referencing this value belong to this capability's functional block.
priority	string	No	Business priority. One of: critical, high, medium, low.
requirements	string[]	No	Traceable requirement identifiers or descriptions this capability satisfies.

YAML example:

```

entities:
  capabilities:
    - id: CAP-F1
      name: User Authentication
      status: ACTIVE
      f_block: F1
      priority: critical
      requirements:
        - "REQ-SEC-001: All users must authenticate before accessing resources"
        - "REQ-SEC-002: Support multi-factor authentication"
    - id: CAP-F2
      name: Order Processing
      status: ACTIVE
      f_block: F2
      priority: high
  
```

1.5.3.3 4.3.3 Behaviors A Behavior is a workflow, use case, or operational sequence that the system performs. Behaviors describe how the system responds to triggers and which steps it executes. They capture the dynamic aspects of the architecture.

Type-specific fields:

Field	Type	Required?	Description
trigger	string	No	The event or condition that initiates this behavior.
actor	string	No	Entity ID of the Actor that initiates this behavior.
preconditions	string[]	No	Conditions that MUST be true before this behavior can execute.
postconditions	string[]	No	Conditions that MUST be true after successful execution.
steps	string[]	No	Ordered list of steps in the behavior's main flow.
frequency	string	No	How often this behavior is executed (e.g., on-demand, hourly, continuous).
priority	string	No	Execution priority. One of: critical, high, medium, low.
pattern	string	No	Behavioral execution pattern. One of: sequential, event-driven, state-machine, saga, pipeline, parallel.
states	string[]	No	For state-machine patterns, the ordered list of states.
compensations	string[]	No	For saga patterns, the compensation actions for rollback.

YAML example:

```

entities:
  behaviors:
    - id: BHV-1
      name: User Login
      status: ACTIVE
      trigger: "User submits credentials"
      actor: ACT-1
      pattern: sequential
      preconditions:
        - "User has registered account"
      postconditions:
        - "Session token is issued"
        - "Last login timestamp is updated"
      steps:
        - "Validate credentials against identity store"
        - "Generate session token"
        - "Record login event"
      frequency: on-demand
      priority: critical
    - id: BHV-2
      name: Order Fulfillment
      status: ACTIVE
      trigger: "Order placed"
      pattern: saga
      steps:
        - "Reserve inventory"

```

- "Charge payment"
- "Dispatch shipment"

compensations:

- "Release inventory reservation"
- "Refund payment"
- "Cancel shipment"

1.5.3.4 4.3.4 Interfaces An Interface is an API, protocol, or data exchange boundary between components or between the system and external agents. Interfaces define the contracts through which entities communicate.

Type-specific fields:

Field	Type	Required?	Description
type	string	Yes	Interface type. One of: REST, WebSocket, database, file, message-queue, internal, external.
protocol	string	No	Communication protocol (e.g., HTTP/2, AMQP, gRPC, Modbus).
provider	string	No	Entity ID of the component that exposes this interface.
consumer	string	No	Entity ID of the component that consumes this interface. For interfaces with multiple consumers, use relationships instead.
data_format	string	No	Data serialization format (e.g., JSON, Protobuf, XML, CSV).
endpoints	string[]	No	List of endpoint paths or addresses.
schema	string	No	Reference to an external schema definition (e.g., OpenAPI spec path, JSON Schema URI).

YAML example:

```
entities:
  interfaces:
    - id: IF-1
      name: Authentication API
      status: ACTIVE
      type: REST
      protocol: HTTP/2
      provider: COMP-AUTH
      data_format: JSON
      endpoints:
        - "POST /api/v1/auth/login"
        - "POST /api/v1/auth/logout"
        - "POST /api/v1/auth/refresh"
      schema: "openapi/auth-api.yaml"
```

- id: IF-2
name: Order Events
status: ACTIVE
type: message-queue
protocol: AMQP
data_format: JSON

1.5.3.5 4.3.5 Constraints A Constraint is a non-functional requirement, design rule, or limitation that applies to one or more entities. Constraints capture the quality attributes, regulatory requirements, and design decisions that bound the solution space.

Type-specific fields:

Field	Type	Required?	Description
type	string	Yes	Constraint category. One of: performance, security, reliability, scalability, regulatory, technology, operational, failure-mode.
metric	string	No	The measurable quantity this constraint governs (e.g., response_time, availability, throughput).
threshold	string	No	The threshold value for the metric (e.g., < 200ms, >= 99.9%, <= 1000 req/s).
rationale	string	No	Explanation of why this constraint exists.

YAML example:

```
entities:
  constraints:
    - id: CON-1
      name: API Response Time
      status: ACTIVE
      type: performance
      metric: response_time_p99
      threshold: "< 200ms"
      rationale: "User research shows abandonment above 200ms for checkout flow"
    - id: CON-2
      name: Data Encryption at Rest
      status: ACTIVE
      type: security
      rationale: "GDPR Article 32 requires appropriate technical measures"
    - id: CON-3
      name: Single Region Failure
      status: ACTIVE
      type: failure-mode
      metric: availability
      threshold: ">= 99.95%"
      rationale: "SLA commitment to enterprise customers"
```

1.5.3.6 4.3.6 Layers A Layer is an architectural tier representing a horizontal slice of the system organized by abstraction level, deployment concern, or functional grouping. Components are assigned to layers. The order field establishes the dependency direction: lower-numbered layers are closer to the user or system boundary; higher-numbered layers are closer to data or infrastructure.

Type-specific fields:

Field	Type	Required?	Description
order	integer	No	Numeric ordering of this layer relative to others. Lower values are closer to the system boundary.
technology	string[]	No	Technologies used in this layer.
directories	string[]	No	Source directories belonging to this layer.

YAML example:

```
entities:
  layers:
    - id: LYR-WEB
      name: Web Layer
      status: ACTIVE
      order: 1
      technology:
        - "React"
        - "TypeScript"
      directories:
        - "src/web/"
        - "src/components/"
    - id: LYR-SVC
      name: Service Layer
      status: ACTIVE
      order: 2
      technology:
        - "Python"
        - "FastAPI"
      directories:
        - "src/services/"
    - id: LYR-DATA
      name: Data Layer
      status: ACTIVE
      order: 3
      technology:
        - "PostgreSQL"
        - "SQLAlchemy"
      directories:
        - "src/models/"
        - "src/repositories/"
```

1.5.3.7 4.3.7 Components A Component is a deployable unit, module, package, or physical part that constitutes the system. Components are the most detailed structural entity and carry implementation-level information including file references, function signatures, constants, and test contracts.

Components are the primary bridge between the architecture model and source code. The `files`, `symbols`, `functions`, `constants`, `signatures`, and `test_contracts` fields enable round-trip traceability and code regeneration.

Type-specific fields:

Field	Type	Required?	Description
<code>layer</code>	<code>string</code>	No	Entity ID of the Layer this component belongs to.
<code>f_block</code>	<code>string</code>	No	F-block identifier linking this component to a Capability.
<code>technology</code>	<code>string</code>	No	Primary technology or language (e.g., Python, TypeScript, C++).
<code>files</code>	<code>string[]</code>	No	Source file paths belonging to this component.
<code>responsibilities</code>	<code>string[]</code>	No	High-level descriptions of what this component does.
<code>kind</code>	<code>string</code>	No	Component kind. One of: <code>module</code> , <code>package</code> , <code>service</code> , <code>library</code> , <code>cli</code> , <code>data-model</code> , <code>data-store</code> , <code>infrastructure</code> , <code>framework</code> , <code>ui</code> , <code>pipeline</code> .
<code>fields</code>	<code>object[]</code>	No	For data-model components, the data fields with name, type, and constraints.
<code>region</code>	<code>string</code>	No	Deployment region or zone (e.g., <code>us-east-1</code> , <code>eu-west-1</code>).
<code>replicas</code>	<code>integer</code>	No	Number of deployed replicas.
<code>symbols</code>	<code>string[]</code>	No	Exported symbol names (classes, functions, constants) for interface contracts.
<code>functions</code>	<code>object[]</code>	No	Function-level detail. Each object: <code>{name, signature, body_hint, docstring}</code> .
<code>constants</code>	<code>object[]</code>	No	Module-level constants. Each object: <code>{name, value, type}</code> .
<code>signatures</code>	<code>object[]</code>	No	Detailed function signatures. Each object: <code>{name, params[], return_type, raises[]}</code> .
<code>test_contracts</code>	<code>object[]</code>	No	Test-derived behavioral contracts. Each object: <code>{test_name, asserts, description}</code> .
<code>observability</code>	<code>string[]</code>	No	Observability endpoints or integrations (e.g., <code>prometheus:/metrics</code> , <code>jaeger</code>).

YAML example:

`entities:`

```

components:
- id: COMP-AUTH
  name: Authentication Service
  status: ACTIVE
  layer: LYR-SVC
  f_block: F1
  technology: Python
  kind: service
  files:
    - "src/services/auth.py"
    - "src/services/token.py"
  responsibilities:
    - "Validate user credentials"
    - "Issue and refresh JWT tokens"
  functions:
    - name: authenticate
      signature: "(username: str, password: str) -> Token"
      body_hint: "validate credentials, generate JWT"
    - name: refresh_token
      signature: "(token: str) -> Token"
  constants:
    - name: TOKEN_EXPIRY_SECONDS
      value: "3600"
      type: int
  test_contracts:
    - test_name: test_authenticate_valid_credentials
      asserts: "returns Token with valid JWT"
    - test_name: test_authenticate_invalid_password
      asserts: "raises AuthenticationError"
  observability:
    - "prometheus:/metrics"
- id: COMP-REPO
  name: User Repository
  status: ACTIVE
  layer: LYR-DATA
  f_block: F1
  technology: Python
  kind: data-model
  files:
    - "src/models/user.py"
    - "src/repositories/user_repo.py"

```

1.5.3.8 4.3.8 Systems A System is a logical subsystem or bounded context within the architecture. It groups components into a cohesive unit that MAY be further decomposed into a sub-model (see Section 5). Systems provide an intermediate level of abstraction between capabilities and components.

Type-specific fields:

Field	Type	Required?	Description
layer	string	No	Entity ID of the Layer this system primarily resides in.

Field	Type	Required?	Description
f_block	string	No	F-block identifier linking this system to a Capability.
complexity_score	number	No	Numeric complexity score (0.0 - 1.0) indicating the relative complexity of this subsystem.
sub_model_ref	string	No	Relative path to a sub-model <code>.architecture-model.yaml</code> that decomposes this system. See Section 5.
component_ids	string[]	No	Entity IDs of components belonging to this system.

YAML example:

```

entities:
  systems:
    - id: SYS-IAM
      name: Identity and Access Management
      status: ACTIVE
      layer: LYR-SVC
      f_block: F1
      complexity_score: 0.7
      component_ids:
        - COMP-AUTH
        - COMP-REPO
        - COMP-RBAC
    - id: SYS-ORDER
      name: Order Management
      status: ACTIVE
      f_block: F2
      sub_model_ref: "subsystems/order-management/.architecture-model.yaml"

```

1.5.3.9 4.3.9 Data A Data entity represents a data structure, schema, bill of materials, or information asset managed by the system. Data entities capture the shape and sensitivity of information flowing through or stored by the system.

Type-specific fields:

Field	Type	Required?	Description
schema_def	string	No	Inline schema definition or reference to external schema file.
format	string	No	Data format (e.g., JSON, Protobuf, SQL, CSV, binary).
fields	object[]	No	Field definitions. Each object: {name, type, required, description}.
owner	string	No	Entity ID of the component or team that owns this data.

Field	Type	Required?	Description
sensitivity	string	No	Data sensitivity classification (e.g., public, internal, confidential, restricted).

YAML example:

```

entities:
  data:
    - id: DATA-USER
      name: User Profile
      status: ACTIVE
      format: JSON
      sensitivity: confidential
      owner: COMP-REPO
      fields:
        - name: user_id
          type: uuid
          required: true
          description: "Unique user identifier"
        - name: email
          type: string
          required: true
          description: "User email address (PII)"
        - name: roles
          type: string[]
          required: false
          description: "Assigned role identifiers"
    - id: DATA-ORDER
      name: Order Record
      status: ACTIVE
      format: JSON
      sensitivity: internal

```

1.5.3.10 4.3.10 Events An Event is a signal, message, command, or notification that flows between entities. Events capture the asynchronous communication patterns in the system.

Type-specific fields:

Field	Type	Required?	Description
kind	string	Yes	Event kind. One of: message, signal, command, notification, alarm.
source	string	No	Entity ID of the event producer.
target	string	No	Entity ID of the event consumer. For broadcast events, use relationships instead.
payload	string	No	Description of the event payload structure or reference to a Data entity.

Field	Type	Required?	Description
frequency	string	No	Expected event frequency (e.g., rare, occasional, frequent, continuous).
reliability	string	No	Delivery guarantee (e.g., at-most-once, at-least-once, exactly-once).

YAML example:

```

entities:
  events:
    - id: EVT-ORDER-PLACED
      name: Order Placed
      status: ACTIVE
      kind: command
      source: COMP-AUTH
      payload: "DATA-ORDER"
      frequency: frequent
      reliability: exactly-once
    - id: EVT-PAYMENT-FAILED
      name: Payment Failed
      status: ACTIVE
      kind: notification
      source: ACT-2
      frequency: occasional
      reliability: at-least-once
    - id: EVT-TEMP-ALARM
      name: Temperature Alarm
      status: ACTIVE
      kind: alarm
      frequency: rare

```

1.5.3.11 4.3.11 Resources A Resource is an external dependency that the system requires but does not own. Resources represent databases, external APIs, hardware devices, storage systems, and other infrastructure that the system consumes.

Type-specific fields:

Field	Type	Required?	Description
kind	string	Yes	Resource kind. One of: database, api, hardware, storage, compute, sensor, actuator.
provider	string	No	Provider name or organization (e.g., AWS, Siemens, internal).
location	string	No	Network address, URI, or physical location of the resource.
sla	string	No	Service-level agreement or availability guarantee.

YAML example:

```
entities:
  resources:
    - id: RES-DB
      name: Primary Database
      status: ACTIVE
      kind: database
      provider: AWS
      location: "rds.us-east-1.amazonaws.com"
      sla: "99.99% availability"
    - id: RES-REDIS
      name: Session Cache
      status: ACTIVE
      kind: database
      provider: AWS
      location: "elasticsearch.us-east-1.amazonaws.com"
    - id: RES-TEMP-SENSOR
      name: Process Temperature Sensor
      status: ACTIVE
      kind: sensor
      provider: Siemens
      location: "Reactor vessel, port T-14"
```

1.5.3.12 4.3.12 Environments An Environment is a deployment target or operational context in which the system or its components run. Environments capture the infrastructure, constraints, and configuration that differ between development, testing, and production.

Type-specific fields:

Field	Type	Required?	Description
kind	string	Yes	Environment kind. One of: development, staging, production, test, field, laboratory.
infrastructure	string[]	No	Infrastructure components in this environment (e.g., kubernetes, docker, bare-metal).
constraints	string[]	No	Environment-specific constraints or limitations.
region	string	No	Geographic region or deployment zone.

YAML example:

```
entities:
  environments:
    - id: ENV-PROD
      name: Production
      status: ACTIVE
      kind: production
      region: us-east-1
```

```

    infrastructure:
      - "kubernetes"
      - "istio"
      - "prometheus"
    constraints:
      - "All traffic must be encrypted in transit"
      - "No direct database access from external networks"
  - id: ENV-DEV
    name: Development
    status: ACTIVE
    kind: development
    infrastructure:
      - "docker-compose"

```

1.5.3.13 4.3.13 Quality Attributes A Quality Attribute is a measured or measurable property of the system that expresses a quality concern. Unlike Constraints (which express requirements), Quality Attributes capture both the target and the current measured value, enabling gap analysis.

Type-specific fields:

Field	Type	Required?	Description
metric	string	Yes	The metric being measured (e.g., latency_p99, test_coverage, mtbf).
target	string	No	Target value for this metric (e.g., < 100ms, >= 80%).
current	string	No	Current measured value (e.g., 87ms, 76%).
measurement_method	string	No	How this metric is measured (e.g., prometheus query, coverage report, load test).
applies_to	string[]	No	Entity IDs this quality attribute applies to.

YAML example:

```

entities:
  quality_attributes:
    - id: QA-LATENCY
      name: API Latency
      status: ACTIVE
      metric: latency_p99
      target: "< 100ms"
      current: "87ms"
      measurement_method: "Prometheus: histogram_quantile(0.99, http_request_duration)"
      applies_to:
        - COMP-AUTH
        - IF-1
    - id: QA-COVERAGE
      name: Test Coverage
      status: ACTIVE

```

```
metric: line_coverage
target: ">= 80%"
current: "76%"
measurement_method: "pytest-cov report"
```

1.5.3.14 4.3.14 Decisions A Decision is an Architecture Decision Record (ADR) capturing a significant design choice, its context, the options considered, and the rationale for the chosen option. Decisions provide traceability for why the architecture is shaped the way it is.

Type-specific fields:

Field	Type	Required?	Description
decision_status	string	Yes	Status of this decision. One of: proposed, accepted, deprecated, superseded.
context	string	No	The situation or problem that motivated this decision.
options	string[]	No	The options that were considered.
rationale	string	No	Explanation of why the chosen option was selected.
consequences	string[]	No	Known consequences (positive and negative) of this decision.
supersedes	string	No	Entity ID of the Decision this one supersedes.

YAML example:

```
entities:
  decisions:
    - id: ADR-001
      name: Use JWT for Session Tokens
      status: ACTIVE
      decision_status: accepted
      context: "Need stateless authentication for horizontally-scaled services"
      options:
        - "Server-side sessions with Redis"
        - "JWT tokens with short expiry"
        - "OAuth2 with external IdP"
      rationale: "JWT allows stateless verification without shared session store"
      consequences:
        - "Positive: No session store dependency"
        - "Positive: Easy horizontal scaling"
        - "Negative: Token revocation requires deny-list"
    - id: ADR-002
      name: Migrate to Event-Driven Order Processing
      status: ACTIVE
      decision_status: proposed
      context: "Synchronous order processing causes cascading failures"
      supersedes: ADR-000
```

1.5.3.15 4.3.15 Lifecycles A Lifecycle entity tracks the version, phase, and temporal progression of the system or its components. Lifecycles capture when entities transition between phases and support migration planning.

Type-specific fields:

Field	Type	Required?	Description
phase	string	Yes	Current lifecycle phase. One of: concept, design, prototype, development, testing, production, maintenance, end-of-life.
version	string	No	Version identifier (e.g., 1.0.0, 2.0-rc1).
start_date	string (ISO 8601)	No	Date this phase began.
end_date	string (ISO 8601)	No	Date this phase ended or is planned to end.
migration_from	string	No	Entity ID or version identifier of the predecessor.
migration_to	string	No	Entity ID or version identifier of the successor.
milestones	object[]	No	Key milestones. Each object: {name, date, description}.

YAML example:

```
entities:
  lifecycles:
    - id: LC-V2
      name: Platform v2.0
      status: ACTIVE
      phase: development
      version: "2.0.0"
      start_date: "2026-01-15"
      migration_from: LC-V1
      milestones:
        - name: Architecture Review
          date: "2026-02-01"
          description: "Complete architecture model and peer review"
        - name: Beta Release
          date: "2026-06-01"
          description: "Feature-complete beta for internal testing"
    - id: LC-V1
      name: Platform v1.0
      status: DEPRECATED
      phase: maintenance
      version: "1.9.3"
      migration_to: LC-V2
      end_date: "2026-12-31"
```

1.5.4 4.4 Relationships

Relationships are directed, typed links between entities. They form the edges of the architecture graph and express how entities interact, depend on, or constrain each other.

Every relationship MUST reference existing entity IDs in its from and to fields. Relationships are listed as a flat array under the top-level relationships key.

Relationship properties:

Field	Type	Required?	Description
type	string	Yes	Relationship type. One of the 16 types defined below.
from	string	Yes	Entity ID of the source entity.
to	string	Yes	Entity ID of the target entity.
description	string	No	Human-readable description of this specific relationship.
strength	string	No	Coupling strength. One of: strong, moderate, weak. Default: strong.
extensions	object	No	Arbitrary key-value pairs for domain-specific relationship metadata.

1.5.4.1 4.4.1 Structural Relationships These relationships describe the static structure of the system.

Type	Semantics	Typical From → To
realizes	The source entity implements or provides the target capability.	Component → Capability
contains	The source entity structurally contains the target entity.	Layer → Component, System → Component
depends-on	The source entity requires the target entity to function.	Component → Component, Component → Resource
exposes	The source entity makes the target interface available.	Component → Interface
consumes	The source entity uses the target interface or resource.	Component → Interface, Component → Resource

1.5.4.2 4.4.2 Traceability Relationships These relationships link architectural elements to requirements, constraints, and deployment targets.

Type	Semantics	Typical From → To
traces-to	The source entity traces to a requirement, decision, or external artifact.	Component → Decision, Capability → Data
allocated-to	The source entity is allocated to the target environment or resource.	Component → Environment, Component → Resource

Type	Semantics	Typical From → To
constrained-by	The source entity is constrained by the target constraint or quality attribute.	Component → Constraint, Interface → Constraint

1.5.4.3 4.4.3 Spatial Relationships These relationships describe physical arrangement and connectivity. They are primarily used with controls, mechanical, and electrical domain profiles (Section 7) but MAY be used in any domain.

Type	Semantics	Typical From → To
mounted-on	The source entity is physically mounted on the target.	Component → Component
connected-at	The source entity has a physical connection at the target.	Interface → Component
routed-through	The source signal or flow is routed through the target.	Interface → Component, Event → Component

1.5.4.4 4.4.4 Data and Event Flow Relationships These relationships describe how data and events flow through the system.

Type	Semantics	Typical From → To
produces	The source entity produces the target data or event.	Component → Event, Component → Data
subscribes-to	The source entity subscribes to the target event.	Component → Event
transforms	The source entity transforms the target data.	Component → Data

1.5.4.5 4.4.5 Lifecycle Relationships These relationships describe temporal succession and migration paths.

Type	Semantics	Typical From → To
supersedes	The source entity replaces the target entity.	Decision → Decision, Lifecycle → Lifecycle
migrates-to	The source entity will be migrated to the target.	Component → Component, Lifecycle → Lifecycle

YAML example:

```
relationships:
- type: realizes
  from: COMP-AUTH
  to: CAP-F1
  description: "Authentication service implements user authentication capability"
- type: exposes
  from: COMP-AUTH
```

- to: IF-1
- type: depends-on
 - from: COMP-AUTH
 - to: RES-DB
 - strength: strong
- type: constrained-by
 - from: COMP-AUTH
 - to: CON-1
- type: produces
 - from: COMP-AUTH
 - to: EVT-ORDER-PLACED
- type: allocated-to
 - from: COMP-AUTH
 - to: ENV-PROD
- type: supersedes
 - from: ADR-002
 - to: ADR-001

1.5.5 4.5 Base Entity Properties

Every entity, regardless of type, inherits the following base properties.

Field	Type	Required?	Description
id	string	Yes	Unique identifier within the model. MUST be unique across all entity types. Recommended format: TYPE-name (e.g., COMP-AUTH, IF-1, CAP-F1).
name	string	Yes	Human-readable name for the entity.
status	string	Yes	Lifecycle status. One of: ACTIVE, PLANNED, DORMANT, DEPRECATED.
description	string	No	Free-text description of the entity's purpose and behavior.
tags	string[]	No	Arbitrary tags for categorization and filtering.
source_file	string	No	Path to the source file where this entity is defined or implemented.
source_line	integer	No	Line number in source_file where this entity begins.
extensions	object	No	Arbitrary key-value pairs for domain-specific or tool-specific metadata not covered by the standard fields.

ID uniqueness rule: Entity IDs MUST be unique across the entire model, not just within their entity type. That is, a Component and an Interface MUST NOT share the same id value.

Extensions: The extensions field provides an escape hatch for domain-specific metadata. Conformant implementations MUST preserve extensions during round-trip serialization but are not required to interpret their contents.

1.6 5. Recursive Decomposition

1.6.1 5.1 Overview

Any Component or System entity MAY be decomposed into a full sub-model. This enables recursive architecture description at arbitrary depth, supporting systems-of-systems, product-line architectures, and multi-team decomposition.

1.6.2 5.2 Mechanism

Recursive decomposition uses two fields in the meta section of the sub-model:

- `parent_model`: Relative path from the sub-model to its parent `.architecture-model.yaml`.
- `refines_component`: Entity ID of the parent model's Component or System that this sub-model decomposes.

And one field on the parent model's System entity:

- `sub_model_ref`: Relative path from the parent model to the sub-model.

1.6.3 5.3 Rules

1. The `refines_component` value in the sub-model MUST match an entity ID in the parent model.
2. The `sub_model_ref` value on the parent entity MUST point to a valid `.architecture-model.yaml` file.
3. Interface entities that cross the decomposition boundary MUST use the same `id` in both the parent model and the sub-model. This enables contract enforcement across levels.
4. A sub-model MAY itself contain further sub-models, enabling arbitrary depth.
5. Circular decomposition (A refines B which refines A) is prohibited. Implementations MUST detect and reject cycles.

1.6.4 5.4 Example

Parent model (`.architecture-model.yaml`):

```
meta:
  schema_version: "2.0"
  project: acme-platform
entities:
  systems:
    - id: SYS-ORDER
      name: Order Management
      status: ACTIVE
      sub_model_ref: "subsystems/order/.architecture-model.yaml"
  interfaces:
    - id: IF-ORDER-API
      name: Order API
      status: ACTIVE
      type: REST
relationships: []
```

Sub-model (`subsystems/order/.architecture-model.yaml`):

```
meta:
  schema_version: "2.0"
```

```

project: acme-platform
system: order-management
parent_model: "../../architecture-model.yaml"
refines_component: SYS-ORDER
entities:
  components:
    - id: COMP-ORDER-SVC
      name: Order Service
      status: ACTIVE
      kind: service
  interfaces:
    - id: IF-ORDER-API
      name: Order API
      status: ACTIVE
      type: REST
      provider: COMP-ORDER-SVC
      endpoints:
        - "POST /api/v1/orders"
        - "GET /api/v1/orders/{id}"
  relationships:
    - type: exposes
      from: COMP-ORDER-SVC
      to: IF-ORDER-API

```

Note that IF-ORDER-API appears in both models with the same id, enabling cross-level contract verification.

1.7 6. Reality Manifest

1.7.1 6.1 Overview

The Reality Manifest is a deterministic, AST-derived inventory of a codebase's actual structure. It serves as the ground truth against which architecture models are validated. The manifest is produced by static analysis — no LLM is involved in its generation.

1.7.2 6.2 Purpose

The manifest bridges the gap between architectural intent (the model) and code reality. It enables:

- **Drift detection.** Identifying when the model no longer matches the code.
- **Completeness checking.** Ensuring all source files are accounted for in the model.
- **Enrichment.** Providing AST-level detail (function signatures, constants, class hierarchies) that LLMs use to produce implementation-accurate models.

1.7.3 6.3 Manifest Structure

A Reality Manifest is a structured object with the following top-level sections:

Manifest

```

+--- project_root: string
+--- generated_at: string (ISO 8601)
+--- source_language: string
+--- summary
|   +--- total_files: integer

```

```

|     +-- total_functions: integer
|     +-- total_classes: integer
|     +-- total_lines: integer
+-- files[]
|     +-- path: string
|     +-- functions[]
|         +-- name: string
|         +-- signature: string
|         +-- calls: string[]
|         +-- docstring: string
|         +-- raises: string[]
|     +-- classes[]
|         +-- name: string
|         +-- bases: string[]
|         +-- methods[]
|         +-- attributes[]
|     +-- imports[]
|         +-- module: string
|         +-- names: string[]
|     +-- constants[]
|         +-- name: string
|         +-- value: string
|         +-- type: string
+-- f_blocks[]
|     +-- id: string
|     +-- files: string[]
|     +-- manifest: (recursive per-file detail)

```

1.7.4 6.4 Per-File Detail

For each source file, the manifest records:

Section	Contents
functions	Every function defined in the file: name, full signature, list of called functions, docstring, list of exceptions raised.
classes	Every class: name, base classes, methods (with signatures), attributes.
imports	Every import statement: module path, imported names.
constants	Every module-level constant assignment: name, value, inferred type.

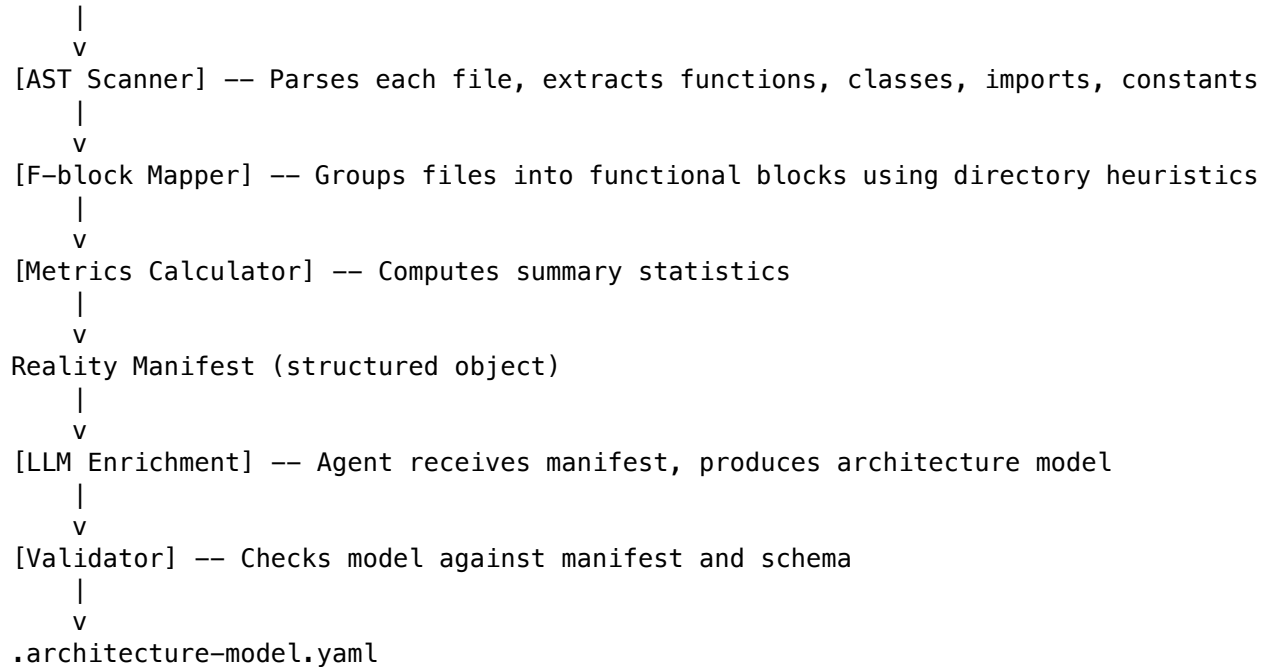
1.7.5 6.5 Per-F-block Detail

For each functional block, the manifest provides a recursive sub-manifest containing the full module-level detail for all files in the F-block. This enables F-block-scoped context slicing (see Section 8).

1.7.6 6.6 Pipeline

The manifest generation pipeline operates as follows:

Source Code



1.7.7 6.7 Manifest Hash

The `manifest_hash` field in the model's meta section (Section 4.2) stores the SHA-256 hash of the serialized manifest. This links a model to a specific code state. Validators SHOULD warn when the current manifest hash does not match the model's recorded hash, indicating potential drift.

1.8 7. Domain Profiles

1.8.1 7.1 Overview

Domain profiles extend the base schema with domain-specific enumeration values, entity properties, and validation rules. The base schema (Section 4) is domain-neutral; profiles add terminology and constraints appropriate to specific engineering disciplines.

1.8.2 7.2 Activation

A domain profile is activated by setting the `domain_profile` field in the model's meta section:

```
meta:
  schema_version: "2.0"
  project: reactor-control
  domain_profile: controls
```

If `domain_profile` is omitted, the software profile is assumed.

1.8.3 7.3 Available Profiles

1.8.3.1 7.3.1 Software (Default) The software profile uses the base schema without modification. All entity types, relationship types, and field values defined in Section 4 are the software profile.

1.8.3.2 7.3.2 Controls The controls profile extends the base schema for industrial control systems, SCADA, and automation.

Additional kind values for Components:

- sensor, actuator, plc, hmi, scada-server, fieldbus, safety-relay

Additional type values for Interfaces:

- Modbus, OPC-UA, PROFINET, EtherCAT, HART, fieldbus

Additional properties on Components:

Field	Type	Description
sil_level	integer (1-4)	Safety Integrity Level per IEC 61508.
scan_rate_ms	integer	PLC scan rate in milliseconds.
signal_type	string	For sensors/actuators: analog-4-20mA, analog-0-10V, digital, thermocouple, RTD.
io_address	string	PLC I/O address (e.g., %IW100, %QX0.0).

Additional validation rules:

- Components with kind: sensor or kind: actuator MUST declare signal_type.
- Components with sil_level MUST have at least one constrained-by relationship to a Constraint of type reliability or failure-mode.

1.8.3.3 7.3.3 Mechanical The mechanical profile extends the base schema for mechanical assemblies, manufactured parts, and physical systems.

Additional kind values for Components:

- part, assembly, sub-assembly, fastener, bearing, seal, housing, shaft, gear

Additional properties on Components:

Field	Type	Description
material	string	Material specification (e.g., AISI 4140, AL 6061-T6).
mass_kg	number	Component mass in kilograms.
tolerance	string	Dimensional tolerance class (e.g., IT7, +/- 0.01mm).
finish	string	Surface finish specification (e.g., Ra 0.8, anodized).
part_number	string	Manufacturer or internal part number.

Additional validation rules:

- Components with kind: part SHOULD declare material.
- mounted-on relationships SHOULD be used to express physical assembly hierarchy.

1.8.3.4 7.3.4 Electrical The electrical profile extends the base schema for PCB assemblies, wiring harnesses, and electrical systems.

Additional kind values for Components:

- pcb, connector, power-supply, regulator, capacitor, resistor, ic, mcu, fpga, harness

Additional properties on Components:

Field	Type	Description
voltage_rating	string	Rated voltage (e.g., 3.3V, 24VDC, 120VAC).
current_rating	string	Rated current (e.g., 500mA, 10A).
power_dissipation	string	Power dissipation (e.g., 2.5W).
package	string	Package type (e.g., S0IC-8, QFP-64, T0-220).
schematic_ref	string	Reference designator on schematic (e.g., U1, R23, C15).

Additional validation rules:

- Components with kind: pcb SHOULD have at least one contains relationship.
- Components with kind: power-supply or kind: regulator MUST declare voltage_rating.
- connected-at relationships SHOULD specify pin or terminal information in the relationship description or extensions.

1.8.4 7.4 Profile Composition

A model MUST declare exactly one domain_profile. For systems that span multiple domains (e.g., an industrial machine with mechanical, electrical, and controls subsystems), use recursive decomposition (Section 5) with each sub-model declaring its own profile:

```
machine/.architecture-model.yaml (domain_profile: mechanical)
machine/controls/.architecture-model.yaml (domain_profile: controls)
machine/electrical/.architecture-model.yaml (domain_profile: electrical)
machine/firmware/.architecture-model.yaml (domain_profile: software)
```

1.8.5 7.5 Extensions for Custom Profiles

The extensions field on entities (Section 4.5) MAY be used for domain-specific properties not covered by the four standard profiles. Custom profiles are not part of this standard but SHOULD follow the same conventions:

- Additional enum values SHOULD be documented.
- Additional properties SHOULD include type and description.
- Additional validation rules SHOULD be machine-enforceable.

1.9 8. LLM Integration Protocol

1.9.1 8.1 Overview

The LLM Integration Protocol defines six verbs for AI agent interaction with architecture models. These verbs provide a structured interface for agents to load, query, modify, and reason about architectural models.

Implementations MAY expose these verbs through any mechanism (MCP tools, function calls, REST API, CLI commands). The protocol defines semantics, not transport.

1.9.2 8.2 Verbs

1.9.2.1 8.2.1 LOAD Purpose: Parse and internalize an architecture model for subsequent operations.

Input: Model source — a file path, YAML string, or structured object.

Output: Parsed model object ready for querying.

Semantics:

1. The implementation **MUST** parse the YAML and validate it against the schema.
2. The implementation **MUST** report validation errors if the model is non-conformant.
3. The implementation **SHOULD** compute summary statistics (entity counts, relationship counts, layer structure).
4. After **LOAD**, the model is available for **QUERY**, **IMPACT**, **VALIDATE**, **UPDATE**, and **PROJECT** operations.

1.9.2.2 8.2.2 QUERY Purpose: Answer structural questions about the loaded model.

Input: A natural language or structured query (e.g., “What components realize capability CAP-F1?”, “List all interfaces exposed by COMP-AUTH”).

Output: The relevant subset of the model that answers the query.

Semantics:

1. The implementation **MUST** support queries by entity ID, entity type, and relationship traversal.
2. The implementation **SHOULD** support natural language queries by mapping them to structured operations.
3. Results **MUST** include entity details and relevant relationships.

1.9.2.3 8.2.3 IMPACT Purpose: Trace change impact through the relationship graph.

Input: An entity ID and a proposed change (e.g., “What is affected if COMP-AUTH is modified?”).

Output: The set of directly and transitively affected entities, grouped by impact type (structural, behavioral, contractual).

Semantics:

1. The implementation **MUST** traverse all relationship types from the specified entity.
2. The implementation **SHOULD** distinguish direct impacts (one hop) from transitive impacts (two or more hops).
3. The implementation **SHOULD** classify impacts by relationship type (e.g., depends-on impacts differ from constrained-by impacts).

1.9.2.4 8.2.4 VALIDATE Purpose: Check the model against validation rules and optionally against a Reality Manifest.

Input: The loaded model and optionally a Reality Manifest.

Output: Validation result including score (0-100), list of issues, and pass/fail determination.

Semantics:

1. The implementation **MUST** check all rules defined in Section 9.
2. The implementation **MUST** return a numeric score from 0 to 100.
3. Each issue **MUST** include a severity (error, warning, info), a rule identifier, and a human-readable message.
4. A model with any error-severity issues **MUST NOT** receive a score above 80.

1.9.2.5 8.2.5 UPDATE Purpose: Propose modifications to the model.

Input: A set of proposed changes (add entity, remove entity, modify field, add relationship, remove relationship).

Output: The updated model, with a diff showing what changed.

Semantics:

1. The implementation **MUST** validate the updated model before accepting changes.
2. The implementation **MUST** reject changes that would violate validation rules.
3. The implementation **SHOULD** provide a preview of validation results before applying changes.

1.9.2.6 8.2.6 PROJECT Purpose: Forecast the effects of planned changes on the system.

Input: A description of planned changes (e.g., “Add a new payment provider”, “Deprecate component COMP-LEGACY”).

Output: A projection including affected entities, required model changes, and risk assessment.

Semantics:

1. The implementation **SHOULD** use **IMPACT** analysis as the basis for projection.
2. The implementation **SHOULD** identify entities that would need status changes.
3. The implementation **MAY** suggest new entities or relationships needed to support the planned change.

1.9.3 8.3 Context Compression

When serving model context to LLMs, implementations **SHOULD** compress the model to fit within token budgets. The standard defines three detail levels:

Level	Description	Typical Use
minimal	Entity IDs, names, and types only. No descriptions, no relationships.	Overview, entity enumeration.
standard	Full entities with descriptions. Relationships included. No implementation detail (no functions, constants, test_contracts).	General architectural reasoning.
full	All fields including implementation detail.	Code generation, detailed analysis.

Context slicing strategies:

1. **By F-block.** Include only entities belonging to a specific functional block and their transitive dependencies.
2. **By Layer.** Include only entities in a specific architectural layer.
3. **By Entity.** Include a specific entity and its immediate neighborhood (directly connected entities).

1.10 9. Validation Rules

1.10.1 9.1 Overview

Validation rules ensure structural correctness and semantic consistency of architecture models. All rules are deterministic and require no LLM involvement. Each rule has a severity level that determines its impact on the validation score.

1.10.2 9.2 Severity Levels

Severity	Score Impact	Description
error	-10 per occurrence	Structural violations that make the model invalid.
warning	-3 per occurrence	Inconsistencies that reduce model quality.
info	-0 per occurrence	Suggestions for improvement.

The validation score starts at 100 and is decremented by the score impact of each issue. The minimum score is 0.

1.10.3 9.3 Rules

1.10.3.1 9.3.1 ID Uniqueness (Error) Rule: Every entity `id` MUST be unique across the entire model.

Check: Collect all entity IDs across all entity types. Report duplicates.

1.10.3.2 9.3.2 Referential Integrity (Error) Rule: Every `from` and `to` value in relationships MUST reference an existing entity ID.

Check: For each relationship, verify that both `from` and `to` IDs exist in the entity index.

1.10.3.3 9.3.3 Orphan Detection (Warning) Rule: Every entity SHOULD participate in at least one relationship.

Check: Identify entities that are not referenced by any relationship (neither as `from` nor as `to`). Actors and Environments are exempt from this rule, as they may exist as context without explicit relationships.

1.10.3.4 9.3.4 Status Consistency (Warning) Rule: An ACTIVE entity SHOULD NOT depend on a DEPRECATED entity.

Check: For each depends-on relationship, verify that if the source entity is ACTIVE, the target entity is not DEPRECATED.

1.10.3.5 9.3.5 Capability Realization (Warning) Rule: Every Capability entity SHOULD be realized by at least one Component.

Check: For each Capability, verify that at least one `realizes` relationship targets it.

1.10.3.6 9.3.6 Meta Completeness (Warning) Rule: The meta section SHOULD include `generated_at`, `source_language`, and `manifest_hash`.

Check: Report missing optional meta fields that improve traceability.

1.10.3.7 9.3.7 Regen Readiness (Info) Rule: Components intended for code regeneration SHOULD include `functions`, `constants`, and `test_contracts`.

Check: For each Component with `kind` in (`module`, `package`, `service`, `library`), report if `functions` or `test_contracts` are empty or absent.

1.10.3.8 9.3.8 Relationship Type Validity (Error) Rule: Every relationship type MUST be one of the 16 defined types.

Check: Validate the `type` field against the enumeration in Section 4.4.

1.10.3.9 9.3.9 Entity Type Validity (Error) Rule: Every entity collection key MUST be a recognized plural entity type name.

Check: Validate entity collection keys against the 15 defined types.

1.10.3.10 9.3.10 Required Fields (Error) Rule: Every entity MUST include id, name, and status. Type-specific required fields MUST be present.

Check: Validate presence of required fields per entity type (see field tables in Section 4.3).

1.10.3.11 9.3.11 Enum Validity (Error) Rule: All enumerated fields MUST contain valid values as defined in this specification.

Check: Validate status, kind, type, phase, decision_status, pattern, priority, strength, and other enum fields against their defined value sets. Domain profile extensions (Section 7) expand valid enum values.

1.10.3.12 9.3.12 Decomposition Integrity (Error) Rule: Recursive decomposition MUST be acyclic and referentially valid.

Check: - If parent_model is set, refines_component MUST also be set. - If sub_model_ref is set on an entity, the referenced file MUST exist. - The decomposition graph MUST be acyclic.

1.11 10. Examples

1.11.1 10.1 Software System

A web application with authentication, order processing, and notification services.

```
meta:
  schema_version: "2.0"
  project: acme-store
  domain_profile: software
  source_language: python
  generated_at: "2026-07-18T10:00:00Z"

entities:
  actors:
    - id: ACT-CUSTOMER
      name: Customer
      status: ACTIVE
      type: human
      goals:
        - "Browse and purchase products"
        - "Track order status"

  capabilities:
    - id: CAP-AUTH
      name: Authentication
      status: ACTIVE
      f_block: F1
      priority: critical
    - id: CAP-ORDERS
      name: Order Management
```

```

    status: ACTIVE
    f_block: F2
    priority: high
- id: CAP-NOTIFY
  name: Notifications
  status: ACTIVE
  f_block: F3
  priority: medium

layers:
- id: LYR-API
  name: API Gateway
  status: ACTIVE
  order: 1
  technology:
    - "FastAPI"
  directories:
    - "src/api/"
- id: LYR-DOMAIN
  name: Domain Services
  status: ACTIVE
  order: 2
  technology:
    - "Python"
  directories:
    - "src/services/"
- id: LYR-PERSIST
  name: Persistence
  status: ACTIVE
  order: 3
  technology:
    - "SQLAlchemy"
    - "PostgreSQL"
  directories:
    - "src/repositories/"

components:
- id: COMP-AUTH-SVC
  name: Auth Service
  status: ACTIVE
  layer: LYR-DOMAIN
  f_block: F1
  kind: service
  technology: Python
  files:
    - "src/services/auth.py"
  responsibilities:
    - "Credential validation"
    - "JWT token lifecycle"
- id: COMP-ORDER-SVC
  name: Order Service
  status: ACTIVE
  layer: LYR-DOMAIN
  f_block: F2

```

```

    kind: service
    technology: Python
    files:
      - "src/services/orders.py"
- id: COMP-NOTIFIER
  name: Notification Service
  status: ACTIVE
  layer: LYR-DOMAIN
  f_block: F3
  kind: service
  technology: Python
  files:
    - "src/services/notifications.py"

interfaces:
- id: IF-AUTH-API
  name: Auth REST API
  status: ACTIVE
  type: REST
  protocol: HTTP/2
  provider: COMP-AUTH-SVC
  data_format: JSON
  endpoints:
    - "POST /api/v1/auth/login"
    - "POST /api/v1/auth/refresh"
- id: IF-ORDER-API
  name: Order REST API
  status: ACTIVE
  type: REST
  provider: COMP-ORDER-SVC
  endpoints:
    - "POST /api/v1/orders"
    - "GET /api/v1/orders/{id}"

constraints:
- id: CON-LATENCY
  name: API Response Time
  status: ACTIVE
  type: performance
  metric: latency_p99
  threshold: "< 200ms"

events:
- id: EVT-ORDER-CREATED
  name: Order Created
  status: ACTIVE
  kind: command
  source: COMP-ORDER-SVC
  frequency: frequent
  reliability: exactly-once

resources:
- id: RES-PG
  name: PostgreSQL Database

```

```

    status: ACTIVE
    kind: database
    provider: AWS
    sla: "99.99%"

decisions:
- id: ADR-JWT
  name: Use JWT for Authentication
  status: ACTIVE
  decision_status: accepted
  rationale: "Stateless authentication for horizontal scaling"

relationships:
- type: realizes
  from: COMP-AUTH-SVC
  to: CAP-AUTH
- type: realizes
  from: COMP-ORDER-SVC
  to: CAP-ORDERS
- type: realizes
  from: COMP-NOTIFIER
  to: CAP-NOTIFY
- type: exposes
  from: COMP-AUTH-SVC
  to: IF-AUTH-API
- type: exposes
  from: COMP-ORDER-SVC
  to: IF-ORDER-API
- type: depends-on
  from: COMP-ORDER-SVC
  to: COMP-AUTH-SVC
  strength: strong
- type: depends-on
  from: COMP-AUTH-SVC
  to: RES-PG
- type: produces
  from: COMP-ORDER-SVC
  to: EVT-ORDER-CREATED
- type: subscribes-to
  from: COMP-NOTIFIER
  to: EVT-ORDER-CREATED
- type: constrained-by
  from: IF-AUTH-API
  to: CON-LATENCY
- type: traces-to
  from: COMP-AUTH-SVC
  to: ADR-JWT

```

1.11.2 10.2 Mechanical Gearbox

A two-stage reduction gearbox with housing, shafts, gears, and bearings.

```

meta:
  schema_version: "2.0"
  project: industrial-gearbox
  system: two-stage-reduction
  domain_profile: mechanical

entities:
  capabilities:
    - id: CAP-REDUCE
      name: Speed Reduction
      status: ACTIVE
      f_block: F1
      priority: critical
      requirements:
        - "REQ-M-001: 50:1 total reduction ratio"
        - "REQ-M-002: Rated torque 1200 Nm output"

  components:
    - id: COMP-HOUSING
      name: Gearbox Housing
      status: ACTIVE
      kind: housing
      extensions:
        material: "GG-25 cast iron"
        mass_kg: 45.0
        finish: "machined and painted"
        part_number: "GB-HSG-001"
    - id: COMP-INPUT-SHAFT
      name: Input Shaft
      status: ACTIVE
      kind: shaft
      extensions:
        material: "AISI 4140"
        mass_kg: 2.1
        tolerance: "IT6"
        part_number: "GB-IS-001"
    - id: COMP-PINION-1
      name: First Stage Pinion
      status: ACTIVE
      kind: gear
      extensions:
        material: "20MnCr5 case-hardened"
        mass_kg: 0.8
        tolerance: "DIN 5"
        part_number: "GB-P1-001"
    - id: COMP-GEAR-1
      name: First Stage Gear
      status: ACTIVE
      kind: gear
      extensions:
        material: "20MnCr5 case-hardened"
        mass_kg: 4.2
        tolerance: "DIN 5"
        part_number: "GB-G1-001"

```

- id: COMP-OUTPUT-SHAFT
 - name: Output Shaft
 - status: ACTIVE
 - kind: shaft
 - extensions:
 - material: "AISI 4340"
 - mass_kg: 5.8
 - tolerance: "IT6"
 - part_number: "GB-OS-001"
- id: COMP-BRG-INPUT
 - name: Input Shaft Bearing
 - status: ACTIVE
 - kind: bearing
 - extensions:
 - part_number: "SKF 6208-2RS"
 - mass_kg: 0.3

constraints:

- id: CON-TORQUE
 - name: Output Torque Rating
 - status: ACTIVE
 - type: performance
 - metric: max_torque_nm
 - threshold: ">= 1200"
- id: CON-TEMP
 - name: Operating Temperature
 - status: ACTIVE
 - type: operational
 - metric: oil_temperature_c
 - threshold: "<= 80"
 - rationale: "Lubricant viscosity degrades above 80C"

lifecycles:

- id: LC-GB-V1
 - name: Gearbox v1.0
 - status: ACTIVE
 - phase: production
 - version: "1.0"
 - start_date: "2024-03-01"

relationships:

- type: realizes
 - from: COMP-INPUT-SHAFT
 - to: CAP-REDUCE
- type: mounted-on
 - from: COMP-PINION-1
 - to: COMP-INPUT-SHAFT
 - description: "Pinion keyed to input shaft"
- type: mounted-on
 - from: COMP-GEAR-1
 - to: COMP-OUTPUT-SHAFT
 - description: "Gear keyed to intermediate shaft"
- type: mounted-on
 - from: COMP-BRG-INPUT

```

    to: COMP-HOUSING
    description: "Bearing pressed into housing bore"
  - type: contains
    from: COMP-HOUSING
    to: COMP-INPUT-SHAFT
  - type: contains
    from: COMP-HOUSING
    to: COMP-OUTPUT-SHAFT
  - type: constrained-by
    from: COMP-OUTPUT-SHAFT
    to: CON-TORQUE

```

1.11.3 10.3 Controls System (SCADA)

A water treatment SCADA system with PLCs, sensors, and HMI.

```

meta:
  schema_version: "2.0"
  project: water-treatment
  system: clarifier-control
  domain_profile: controls

entities:
  actors:
    - id: ACT-OPERATOR
      name: Plant Operator
      status: ACTIVE
      type: human
      goals:
        - "Monitor water quality parameters"
        - "Adjust chemical dosing"

  capabilities:
    - id: CAP-DOSE
      name: Chemical Dosing Control
      status: ACTIVE
      f_block: F1
      priority: critical
      requirements:
        - "REQ-C-001: Maintain pH between 6.5 and 8.5"
        - "REQ-C-002: Automatic dosing response within 5 seconds"

  components:
    - id: COMP-PLC
      name: Clarifier PLC
      status: ACTIVE
      kind: plc
      technology: "Siemens S7-1500"
      extensions:
        scan_rate_ms: 100
        sil_level: 2
    - id: COMP-PH-SENSOR
      name: pH Sensor

```



```

    status: ACTIVE
    kind: sensor
    extensions:
      signal_type: "analog-4-20mA"
      io_address: "%IW100"
  - id: COMP-DOSE-PUMP
    name: Dosing Pump
    status: ACTIVE
    kind: actuator
    extensions:
      signal_type: "analog-4-20mA"
      io_address: "%QW200"
  - id: COMP-HMI
    name: Operator HMI
    status: ACTIVE
    kind: hmi
    technology: "WinCC"

interfaces:
  - id: IF-FIELDBUS
    name: PROFINET Fieldbus
    status: ACTIVE
    type: external
    protocol: PROFINET
  - id: IF-HMI-LINK
    name: HMI Communication
    status: ACTIVE
    type: external
    protocol: OPC-UA

constraints:
  - id: CON-SIL2
    name: SIL 2 Compliance
    status: ACTIVE
    type: reliability
    rationale: "Chemical dosing failure could cause environmental discharge violation"
  - id: CON-RESPONSE
    name: Control Loop Response
    status: ACTIVE
    type: performance
    metric: loop_response_time
    threshold: "<= 5s"

behaviors:
  - id: BHV-PH-CONTROL
    name: pH Control Loop
    status: ACTIVE
    trigger: "pH reading outside setpoint band"
    pattern: sequential
    steps:
      - "Read pH sensor value"
      - "Compare against setpoint (7.0 +/- 0.5)"
      - "Calculate PID output"
      - "Adjust dosing pump speed"

```

```

    - "Log control action"
    frequency: continuous
    priority: critical

resources:
  - id: RES-CHEM-TANK
    name: Chemical Storage Tank
    status: ACTIVE
    kind: storage
    location: "Building 3, Chemical Room"

relationships:
  - type: realizes
    from: COMP-PLC
    to: CAP-DOSE
  - type: depends-on
    from: COMP-PLC
    to: COMP-PH-SENSOR
    strength: strong
  - type: depends-on
    from: COMP-PLC
    to: COMP-DOSE-PUMP
    strength: strong
  - type: connected-at
    from: IF-FIELDBUS
    to: COMP-PLC
    description: "PROFINET port X1"
  - type: connected-at
    from: IF-FIELDBUS
    to: COMP-PH-SENSOR
  - type: consumes
    from: COMP-HMI
    to: IF-HMI-LINK
  - type: exposes
    from: COMP-PLC
    to: IF-HMI-LINK
  - type: constrained-by
    from: COMP-PLC
    to: CON-SIL2
  - type: constrained-by
    from: COMP-PLC
    to: CON-RESPONSE

```

1.11.4 10.4 Electrical PCB Assembly

A motor controller PCB with power supply, microcontroller, and gate driver.

```

meta:
  schema_version: "2.0"
  project: motor-controller
  system: inverter-board
  domain_profile: electrical

```

```

entities:
  capabilities:
    - id: CAP-DRIVE
      name: Motor Drive
      status: ACTIVE
      f_block: F1
      priority: critical
      requirements:
        - "REQ-E-001: Drive 3-phase BLDC motor up to 48V/30A"
        - "REQ-E-002: FOC control at 20kHz PWM frequency"

  components:
    - id: COMP-PCB
      name: Inverter PCB
      status: ACTIVE
      kind: pcb
      extensions:
        part_number: "MC-PCB-001"
    - id: COMP-MCU
      name: Motor Control MCU
      status: ACTIVE
      kind: mcu
      technology: "STM32G474"
      extensions:
        package: "LQFP-64"
        voltage_rating: "3.3V"
        schematic_ref: "U1"
    - id: COMP-GATE-DRV
      name: Gate Driver
      status: ACTIVE
      kind: ic
      extensions:
        package: "S0IC-16"
        voltage_rating: "48V"
        schematic_ref: "U2"
        part_number: "DRV8323RS"
    - id: COMP-PSU
      name: 3.3V Regulator
      status: ACTIVE
      kind: regulator
      extensions:
        voltage_rating: "3.3V"
        current_rating: "500mA"
        package: "S0T-223"
        schematic_ref: "U3"
    - id: COMP-CONN-MOTOR
      name: Motor Connector
      status: ACTIVE
      kind: connector
      extensions:
        current_rating: "30A"
        schematic_ref: "J1"

  interfaces:

```

- id: IF-SPI
 - name: MCU-to-Gate-Driver SPI
 - status: ACTIVE
 - type: internal
 - protocol: SPI
 - provider: COMP-MCU
 - consumer: COMP-GATE-DRV
- id: IF-MOTOR-OUT
 - name: 3-Phase Motor Output
 - status: ACTIVE
 - type: external
 - provider: COMP-GATE-DRV

constraints:

- id: CON-EMC
 - name: EMC Compliance
 - status: ACTIVE
 - type: regulatory
 - rationale: "Must pass EN 55032 Class B conducted emissions"
- id: CON-THERMAL
 - name: Thermal Limit
 - status: ACTIVE
 - type: performance
 - metric: junction_temperature
 - threshold: "<= 125C"
 - rationale: "STM32G474 absolute maximum junction temperature"

resources:

- id: RES-12V-INPUT
 - name: 12-48V DC Input
 - status: ACTIVE
 - kind: hardware
 - provider: "External power supply"

relationships:

- type: realizes
 - from: COMP-MCU
 - to: CAP-DRIVE
 - type: contains
 - from: COMP-PCB
 - to: COMP-MCU
 - type: contains
 - from: COMP-PCB
 - to: COMP-GATE-DRV
 - type: contains
 - from: COMP-PCB
 - to: COMP-PSU
 - type: contains
 - from: COMP-PCB
 - to: COMP-CONN-MOTOR
 - type: depends-on
 - from: COMP-MCU
 - to: COMP-PSU
- description: "MCU powered by 3.3V regulator"

- type: depends-on
from: COMP-GATE-DRV
to: RES-12V-INPUT
 - type: exposes
from: COMP-MCU
to: IF-SPI
 - type: consumes
from: COMP-GATE-DRV
to: IF-SPI
 - type: exposes
from: COMP-GATE-DRV
to: IF-MOTOR-OUT
 - type: connected-at
from: IF-MOTOR-OUT
to: COMP-CONN-MOTOR
description: "Phase U, V, W terminals"
 - type: constrained-by
from: COMP-PCB
to: CON-EMC
 - type: constrained-by
from: COMP-MCU
to: CON-THERMAL
-

1.12 Appendix A: JSON Schema Reference

This appendix provides the JSON Schema representation of the Architecture Model Standard v2.0. Conformant implementations SHOULD use this schema for automated validation.

1.12.1 A.1 Top-Level Schema

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://architecture-model-standard.dev/schema/v2.0",
  "title": "Architecture Model Standard v2.0",
  "type": "object",
  "required": ["meta", "entities", "relationships"],
  "additionalProperties": false,
  "properties": {
    "meta": { "$ref": "#/$defs/Meta" },
    "entities": { "$ref": "#/$defs/Entities" },
    "relationships": {
      "type": "array",
      "items": { "$ref": "#/$defs/Relationship" }
    }
  }
}
```

1.12.2 A.2 Meta Schema

```
{
  "$defs": {
    "Meta": {
```

```

    "type": "object",
    "required": ["schema_version", "project"],
    "properties": {
      "schema_version": { "type": "string", "const": "2.0" },
      "project": { "type": "string" },
      "system": { "type": "string" },
      "generated_at": { "type": "string", "format": "date-time" },
      "source_artifacts": { "type": "array", "items": { "type": "string" } },
      "manifest_hash": { "type": "string" },
      "source_language": { "type": "string" },
      "domain_profile": {
        "type": "string",
        "enum": ["software", "controls", "mechanical", "electrical"],
        "default": "software"
      },
      "parent_model": { "type": "string" },
      "refines_component": { "type": "string" }
    }
  }
}

```

1.12.3 A.3 Base Entity Schema

```

{
  "$defs": {
    "BaseEntity": {
      "type": "object",
      "required": ["id", "name", "status"],
      "properties": {
        "id": { "type": "string" },
        "name": { "type": "string" },
        "status": {
          "type": "string",
          "enum": ["ACTIVE", "PLANNED", "DORMANT", "DEPRECATED"]
        },
        "description": { "type": "string" },
        "tags": { "type": "array", "items": { "type": "string" } },
        "source_file": { "type": "string" },
        "source_line": { "type": "integer" },
        "extensions": { "type": "object" }
      }
    }
  }
}

```

1.12.4 A.4 Relationship Schema

```

{
  "$defs": {
    "Relationship": {
      "type": "object",
      "required": ["type", "from", "to"],
      "properties": {

```

```

    "type": {
      "type": "string",
      "enum": [
        "realizes", "contains", "depends-on", "exposes", "consumes",
        "traces-to", "allocated-to", "constrained-by",
        "mounted-on", "connected-at", "routed-through",
        "produces", "subscribes-to", "transforms",
        "supersedes", "migrates-to"
      ]
    },
    "from": { "type": "string" },
    "to": { "type": "string" },
    "description": { "type": "string" },
    "strength": {
      "type": "string",
      "enum": ["strong", "moderate", "weak"],
      "default": "strong"
    },
    "extensions": { "type": "object" }
  }
}
}
}

```

1.12.5 A.5 Entity Collection Keys

The entities object accepts the following keys, each mapping to an array of entities that extend the base schema with type-specific fields:

```

{
  "$defs": {
    "Entities": {
      "type": "object",
      "additionalProperties": false,
      "properties": {
        "actors": { "type": "array" },
        "capabilities": { "type": "array" },
        "behaviors": { "type": "array" },
        "interfaces": { "type": "array" },
        "constraints": { "type": "array" },
        "layers": { "type": "array" },
        "components": { "type": "array" },
        "systems": { "type": "array" },
        "data": { "type": "array" },
        "events": { "type": "array" },
        "resources": { "type": "array" },
        "environments": { "type": "array" },
        "quality_attributes": { "type": "array" },
        "decisions": { "type": "array" },
        "lifecycles": { "type": "array" }
      }
    }
  }
}

```

The complete JSON Schema with full type-specific field definitions for each entity type is available as a standalone file at `src/architecture_model/spec/schema.json`.

1.13 Appendix B: Relationship Type Matrix

This matrix shows which entity types are valid as from and to for each relationship type. “Any” means any entity type is permitted.

Relationship	Valid from	Valid to
realizes	Component, System	Capability
contains	Layer, System, Component	Component, System
depends-on	Component, System	Component, System, Resource
exposes	Component	Interface
consumes	Component, Actor	Interface, Resource
traces-to	Any	Any
allocated-to	Component, System	Environment, Resource
constrained-by	Any	Constraint, Quality Attribute
mounted-on	Component	Component
connected-at	Interface	Component
routed-through	Interface, Event	Component
produces	Component	Event, Data
subscribes-to	Component	Event
transforms	Component	Data
supersedes	Decision, Lifecycle	Decision, Lifecycle
migrates-to	Component, Lifecycle	Component, Lifecycle

Implementations SHOULD validate relationship endpoints against this matrix and report violations as warnings. Implementations MUST NOT reject models solely based on endpoint type mismatches, as domain profiles (Section 7) may extend valid endpoint combinations.

1.14 Appendix C: Changelog

1.14.1 v2.0 (2026-07-18)

- Added 8 new entity types: Systems, Data, Events, Resources, Environments, Quality Attributes, Decisions, Lifecycles (Section 4.3.8 through 4.3.15).
- Added 8 new relationship types: traces-to, allocated-to, constrained-by, mounted-on, connected-at, routed-through, produces, subscribes-to, transforms, supersedes, migrates-to (Section 4.4).
- Renamed uses to consumes for clarity. exposes added as counterpart.
- Added strength and extensions fields to relationships (Section 4.4).
- Added extensions field to base entity properties (Section 4.5).
- Added source_file and source_line to base entity properties.
- Introduced recursive decomposition with parent_model, refines_component, and sub_model_ref (Section 5).
- Formalized the Reality Manifest specification (Section 6).
- Added domain profiles: software, controls, mechanical, electrical (Section 7).
- Formalized the LLM Integration Protocol with 6 verbs (Section 8).
- Expanded validation rules to 12 (Section 9).
- Added regen readiness validation (Section 9.3.7).

- Added functions, constants, signatures, and test_contracts to Component for code regeneration support.

1.14.2 v1.4 (2026-06-15)

- Added constants, signatures (FunctionSignature), and test_contracts (TestContract) to Component entity.
- Schema version bumped to 1.4.

1.14.3 v1.3 (2026-05-01)

- Added symbols and functions to Component entity.
- Added kind field to Component with 11 enum values.
- Schema version bumped to 1.3.

1.14.4 v1.0 (2026-01-01)

- Initial release with 7 entity types: Actors, Capabilities, Behaviors, Interfaces, Constraints, Layers, Components.
- 8 relationship types: realizes, contains, depends-on, exposes, uses, triggers, implements, constrained-by.
- Basic validation rules.
- LLM context formatting.

End of Architecture Model Standard v2.0 Specification.